

Open-Frame-gen: Real-Time Video Frame Interpolation on Constrained Consumer Hardware

Prithivi Nair
Dept. of ECE
PES University
Bangalore, India
prithivinair05@gmail.com

Prajwal R
Dept. of ECE
PES University
Bangalore, India
ruyketsu.prajwal@gmail.com

Madhav S
Dept. of ECE
PES University
Bangalore, India
madhav022004@gmail.com

Rashmi N Ugarakhod
Dept. of ECE
PES University
Bangalore, India
rashmin@pes.edu

Abstract—Video Frame Interpolation (VFI) generates intermediate frames between two input frames to enhance video smoothness. While RIFE and similar advanced methods deliver impressive visuals, they often fall short of the throughput required for real-time applications on standard consumer hardware. Recent studies, such as VFIMamba, emphasize quality metrics but neglect performance data for typical consumer GPUs, whereas LPVFI prioritizes efficiency yet lacks publicly available source code for validation.

To overcome these limitations, we present Open-Frame-gen, which integrates an ultra-lightweight CNN (Nano, $\sim 90K$ parameters) with a GPU-based rendering pipeline. On an NVIDIA RTX 3050 Laptop GPU, Nano achieves 6–10 ms inference latency and 100–200 FPS at standard resolutions, facilitating real-time VFI on entry-level hardware while utilizing only one-tenth of the VRAM required by RIFE HDv3.

Index Terms—video frame interpolation, real-time inference, lightweight CNN, constrained hardware, gaming, AR/VR, optical flow estimation

I. INTRODUCTION

Display technology has advanced toward higher refresh rates to enhance visual experiences. Modern monitors and mobile devices support 120 Hz, 144 Hz, or even 360 Hz, yet a significant disparity remains between display capabilities and actual application performance. Many real-time applications—high-fidelity games, live AR overlays, and video streaming—struggle to maintain stable high frame rates on consumer-grade hardware, highlighting the potential of real-time frame interpolation to bridge this gap.

While VFI is well-established in offline video processing, consumer devices impose real-time constraints that fundamentally alter requirements. In gaming and AR/VR, the entire pipeline—capture, inference, and display—must complete within milliseconds to prevent noticeable “motion-to-photon” latency. Techniques yielding high PSNR but requiring 100 ms per frame are impractical for real-time use, and such models often necessitate complex software stacks incompatible with consumer “zero-dependency” requirements. We present Open-Frame-gen, a complete real-time VFI system for constrained consumer hardware, with three contributions:

- 1) **Nano Architecture**: An ultra-lightweight CNN ($\sim 90K$ parameters, over $60\times$ smaller than RIFE HDv3) combining depthwise separable convolutions and SE attention to achieve 100–200 FPS on entry-level GPUs.

- 2) **Adaptive Pipeline**: An “Auto-Quality” system that dynamically adjusts inference resolution to maintain target refresh rates regardless of scene complexity.
- 3) **GPU-Resident Engine**: A C++ engine leveraging TensorRT and CUDA-D3D11 interoperability that keeps frames in GPU memory throughout, eliminating CPU round-trips.

II. RELATED WORK

Flow-based warping now dominates VFI. Niklaus et al. [4] introduced adaptive separable convolution, training spatially-varying kernels that implicitly model motion, though performance degrades on large motions typical of game footage. RIFE [1] refines motion patterns incrementally via a compact IFNet, balancing quality and speed effectively. DAIN [5] uses monocular depth for better occlusion handling at significantly higher compute cost.

Recent work diverges into two tracks: quality-focused methods like IFRNet [6] refine intermediate features for sharper output, while VFIMamba [2] applies state space models for improved temporal modeling. LPVFI [3] targets efficiency by modulating compute based on motion magnitude. However, none of these provide empirical evaluations on standard 1080p consumer hardware. Moreover, VFIMamba depends on Linux-only `mamba-ssm` CUDA kernels, and LPVFI requires custom CUTLASS operators, both posing serious compilation barriers on typical Windows platforms.

III. SYSTEM ARCHITECTURE

Open-Frame-gen operates as a tightly integrated pipeline where each component minimizes latency while maintaining correctness. Figure 1 illustrates the overall structure.

A. Capture and Display Pipeline

The capture stage uses DirectX Desktop Duplication to acquire frames directly from the GPU compositor as D3D11 textures, feeding them into CUDA without any CPU memory copy. `SetWindowDisplayAffinity` with `WDA_EXCLUDEFROMCAPTURE` excludes the overlay from capture to prevent feedback loops. The display module renders interpolated frames via a click-through overlay (`WS_EX_LAYERED`, `WS_EX_TRANSPARENT`), with frame

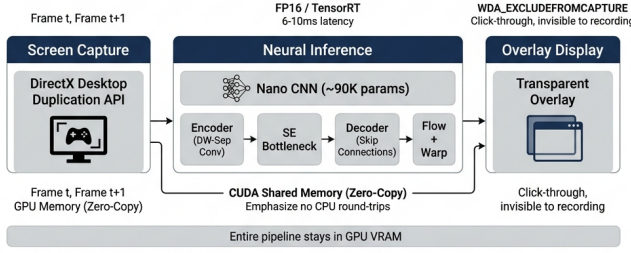


Fig. 1. Open-Frame-gen system architecture: a GPU-resident pipeline where frames flow from DirectX capture through Nano inference to the transparent overlay without any CPU-GPU data transfers.

timing controlled by a hybrid busy-wait loop synchronized to the desktop compositor via `DwmFlush()`.

B. Neural Inference Integration

The Nano model processes consecutive frame pairs to produce intermediate frames. It is exported to ONNX and compiled with TensorRT, enabling FP16 precision and kernel fusion. A single-slot input buffer discards stale frames automatically, keeping queuing latency near zero.

C. Adaptive Resolution Scaling (Auto-Quality)

The “Auto-Quality” mechanism dynamically adapts inference resolution to maintain a target frame rate regardless of transient GPU load. The engine tracks headroom $H = B - L_{\text{last}}$, where $B = 1/\text{FPS}_{\text{target}}$ and L_{last} is the most recent inference latency, following an asymmetric hysteresis controller:

- 1) **Downscale:** If $H < -2$ ms, switch to 80% quality mode (1536×864), reducing pixel count by 36%.
- 2) **Upscale:** If $H > 5$ ms, revert to native 1080p.

The asymmetric thresholds (-2 ms down, $+5$ ms up)—determined empirically across diverse, high-motion gameplay sequences—prevent thrashing under transient load spikes. In practice, quality switches occur fewer than three times per thousand frames.

IV. THE NANO ARCHITECTURE

A. Design Principles

Nano prioritizes temporal latency over peak spatial reconstruction. In interactive applications, delayed high-fidelity frames cause visible lag, while slightly smoothed frames delivered on time preserve fluid motion. The architecture achieves $\sim 90\text{K}$ parameters—over $60\times$ fewer than RIFE HDv3’s 5.66 M—through aggressively reduced channel widths and pervasive depthwise separable convolutions.

B. Network Structure

Nano uses an encoder-decoder design with skip connections, taking concatenated frame pairs as 6-channel RGB input (TABLE I).

The encoder reduces spatial resolution via strided depthwise separable convolutions, expanding channels from 6 to 48,

TABLE I
NANO ARCHITECTURE SUMMARY

Stage	Channels	Resolution
Input (concat frames)	6	$H \times W$
Encoder 1 (DW-Sep, $s=2$)	48	$H/2$
Encoder 2 (DW-Sep, $s=2$)	96	$H/4$
Encoder 3 (DW-Sep, $s=2$)	128	$H/8$
Bottleneck + SE Block	96	$H/8$
Decoder 1 + Skip	96	$H/4$
Decoder 2 + Skip	64	$H/2$
Decoder 3 + Skip	32	$H/2$
Flow Head (3ch)	3	$H/2$
Warp + Blend	3	$H \times W$

96, and 128. Each block applies a depthwise pass (per-channel spatial filtering) followed by a pointwise layer (cross-channel mixing), reducing computation by roughly $C_{\text{out}}/9$ for 3×3 filters—yielding $5\text{--}14\times$ fewer operations per layer. A Squeeze-and-Excitation block at the bottleneck performs global average pooling per channel, passes through two dense layers (reduction factor 8), and rescales channel activations to emphasize motion-relevant features.

The decoder reconstructs spatial detail via bilinear upsampling with skip connections. A specialized flow head outputs two optical flow channels ($\Delta x, \Delta y$) and a blending mask (sigmoid $\in [0, 1]$). Warping uses `grid_sample` with bilinear interpolation, border padding, and `align_corners=True` at quarter-scale, followed by upsampling to full resolution. The model is exported to ONNX (opset 16), where `grid_sample` maps to the native `GridSample` operator supported by TensorRT 10.x without custom plugins. Key deployment settings are listed in Table (TABLE I).

TABLE II
GRID_SAMPLE TENSORRT DEPLOYMENT CONFIGURATION

Parameter	Value
TensorRT Version	10.x
ONNX Opset	16
Interpolation Mode	Bilinear
Padding Mode	Border
<code>align_corners</code>	True
Deployment Method	Native TRT operator

V. TRAINING

A. Dataset

Training uses established public benchmarks: Vimeo-90K [13] (73,171 diverse frames) and UCF101 [11] (fast-motion sequences). For evaluation, all Vimeo-90K test triplets are ranked by average absolute grayscale difference between frames $t - 1$ and $t + 1$; the 300 highest-motion sequences are selected and supplemented with Middlebury [12] and UCF101, producing a challenging benchmark that closely mirrors real-world gameplay conditions.

TABLE III
COMPARISON WITH STATE-OF-THE-ART AT 1920×1080

Method	Params	FPS*	PSNR
VFIMamba-S	16.8 M	1.1 ± 0.1	30.12
RIFE HDv3	5.66 M	8.4 ± 0.2	29.27
Nano (Ours)	89.9 K	101.1 ± 2.4	28.40

*RTX 3050 Laptop GPU (4GB); FP16 (Nano), FP32 (baselines).

B. Loss Function and Training Procedure

The training objective combines three terms:

$$\mathcal{L} = \mathcal{L}_{rec} + \lambda_{tv}\mathcal{L}_{tv} + \lambda_{lap}\mathcal{L}_{lap} \quad (1)$$

$\mathcal{L}_{rec}$ uses L_1 loss for edge preservation. $\mathcal{L}_{tv}$ promotes flow field smoothness, with λ_{tv} annealed from 0.01 to 0.001 over 50 epochs. $\mathcal{L}_{lap}$ (fixed $\lambda_{lap} = 1.0$) enforces multi-scale consistency via a Laplacian pyramid. Optimization uses AdamW ($lr = 2 \times 10^{-4}$, weight decay 10^{-4}), batch size 8, cosine annealing over 100 epochs. Inputs are randomly cropped to 512×512 with horizontal/vertical flipping and temporal reversal augmentation. Mixed FP16/FP32 training reduces memory overhead by approximately 40%.

VI. EXPERIMENTAL EVALUATION

A. Experimental Setup

All experiments run on an NVIDIA RTX 3050 Laptop GPU (4GB VRAM), representative of the xx50/xx60-tier hardware used by the majority of PC gamers. Latency values measure *model-only* inference using `torch.cuda.synchronize()` barriers and `time.perf_counter()`, averaged over 100 timed iterations (Nano) and 20 iterations (RIFE) after warm-up. RIFE HDv3 and VFIMamba-S were evaluated using official checkpoints under PyTorch 2.1 with CUDA 12.1, without TensorRT or CUDA Graphs. Nano uses FP16 precision and CUDA Graph execution. LPVFI [3] was excluded due to unavailable source code.

B. Comparison with State-of-the-Art Methods

This comparison evaluates *practical deployability on constrained hardware*, not peak architectural capability. At 1080p on the RTX 3050, VFIMamba-S managed only 1.1 FPS (905 ms/frame), while the full VFIMamba model repeatedly OOM-crashed. RIFE HDv3 reached 8.4 FPS—a $12\times$ improvement over VFIMamba but still far below real-time thresholds. Nano achieves 101.1 FPS with under 200 MB VRAM, the only method sustaining true real-time throughput (TABLE III).

The quality gap is modest: Nano trails VFIMamba by 1.72 dB PSNR but achieves comparable SSIM to RIFE (0.9525 vs. 0.9515) with a $63\times$ parameter reduction. On the full uncurated Vimeo-90K test set, Nano attains ~ 31.8 dB vs. RIFE’s ~ 35.2 dB, confirming structural coherence across diverse content.

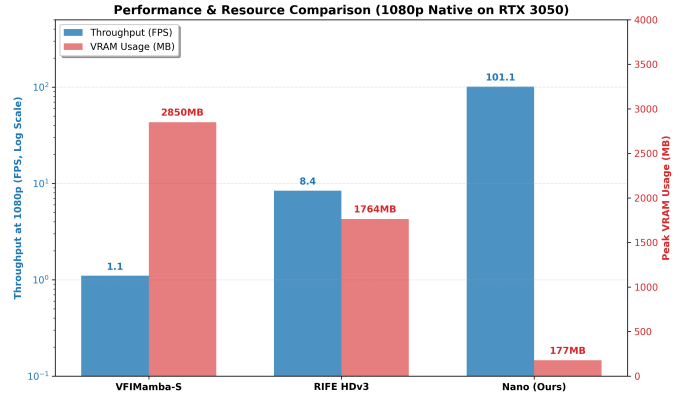


Fig. 2. Performance comparison: only Nano achieves real-time (>60 FPS) on consumer GPU hardware.

TABLE IV
THROUGHPUT ACROSS RESOLUTIONS (RTX 3050 LAPTOP)

Resolution	Latency [†]	FPS	VRAM
720p (1280×720)	4.48 ± 0.12 ms	223.2	81 MB
900p (1600×900)	7.38 ± 0.25 ms	135.5	121 MB
1080p 80% (1536×864)	6.58 ± 0.21 ms	152.0	116 MB
1080p (1920×1080)	9.89 ± 0.45 ms	101.1	177 MB
1440p (2560×1440)	17.84 ± 0.82 ms	56.0	308 MB

[†]Model-only inference latency.

C. Resolution Scaling

At 720p, Nano exceeds 223 FPS, sufficient for $3\times$ frame multiplication under a 60 Hz limit. The 80% quality mode (1536×864) provides the best balance: 152 FPS and 116 MB VRAM, clearing 144 Hz targets with near-native visual quality (TABLE IV).

D. Visual Quality Analysis

TABLE V summarizes full quality metrics. Nano’s slightly elevated tOF (0.67 vs. RIFE’s 0.58) reflects its stateless design; at >100 FPS, temporal flicker from single-frame processing falls below the human perceptual threshold [14]. Notably, Nano’s SSIM marginally exceeds RIFE’s (0.9525 vs. 0.9515) despite lower PSNR—depthwise separable convolutions preserve structural edges at the cost of fine-texture sharpness, a favorable trade-off for gaming content.

TABLE V
VISUAL QUALITY METRICS ON BENCHMARK DATASETS

Method	PSNR [†]	SSIM [†]	LPIPS [↓]	tOF [↓]
VFIMamba [†]	30.12	0.9621	0.031	—
RIFE HDv3	29.27	0.9515	0.045	0.58
Nano (Ours)	28.40	0.9525	0.052	0.67

[†]Published values on Vimeo90K; tOF not reported.

E. Ablation Study

TABLE VI shows cumulative optimization impact. Depthwise separable convolutions provide the largest gain, halving

Throughput and VRAM Scaling Across Resolutions
(RTX 3050 Laptop GPU)

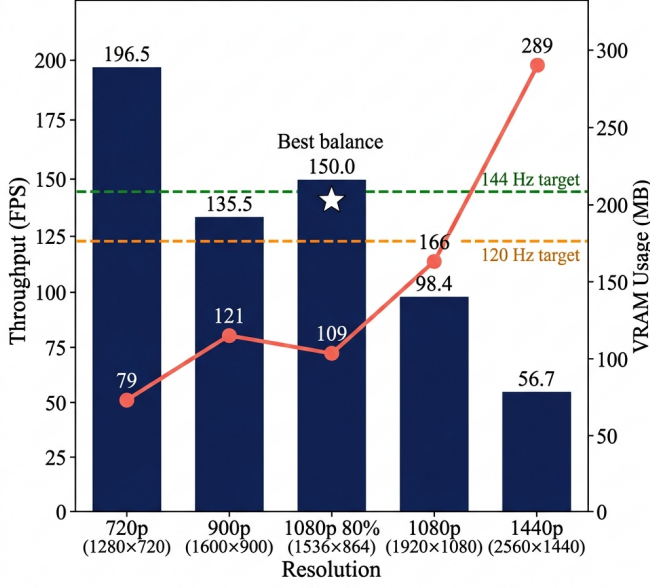


Fig. 3. Throughput and VRAM scaling. The 80% quality mode offers 152 FPS at only 116 MB VRAM.

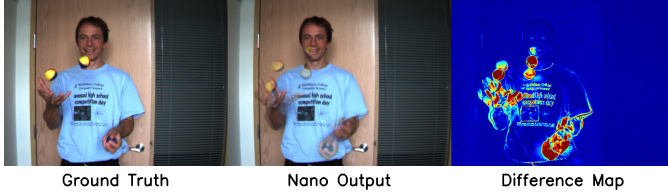


Fig. 4. Frame interpolation quality on high-motion gaming content. Top: input frames $t-1$ and $t+1$. Bottom: ground truth t (left) and Nano output (right). PSNR=23.81 dB, SSIM=0.9181.

latency from 14.2 to 6.8 ms. FP16 and tensor reuse contribute incremental improvements. The SE block adds 0.6 ms but recovers 0.4 dB PSNR, a worthwhile trade-off.

F. End-to-End Pipeline Latency

Figure 5 shows full pipeline timing including capture (1–2 ms) and display (1–2 ms) overhead. At 1080p 80% quality mode, total end-to-end latency is 10–12 ms—well within the 16.7 ms budget for 60→120 FPS doubling.

VII. DISCUSSION

A. The Quality-Latency Trade-off

The 0.87 dB PSNR gap between Nano and RIFE reflects a deliberate design priority. Human perception is more sensitive to temporal glitches than spatial blur during motion [14]—a 100 ms delay is more disruptive than mild texture softness when timing stays consistent. At 144 Hz, RIFE’s 118.8 ms lag represents a 17-frame deficit per visual event, whereas Nano’s sub-10 ms latency meets timing requirements even at 100 Hz without resolution scaling. The marginally higher SSIM of

TABLE VI
ABLATION STUDY: CUMULATIVE OPTIMIZATION IMPACT

Configuration	Latency	VRAM
Baseline (standard conv)	14.2 ms	310 MB
+ Depthwise separable	6.8 ms	185 MB
+ FP16 precision	5.4 ms	155 MB
+ Tensor reuse	5.1 ms	150 MB
+ SE block (final)	5.7 ms	166 MB

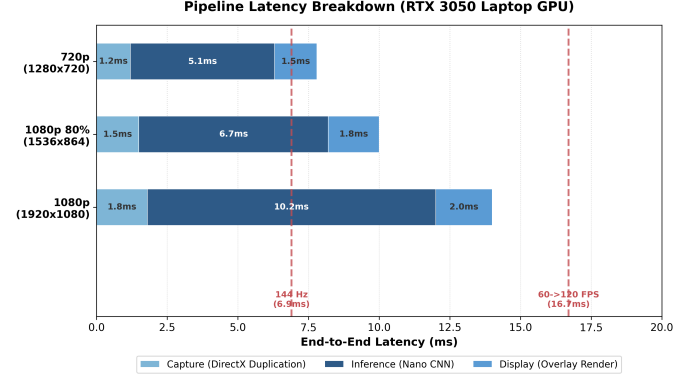


Fig. 5. End-to-end latency breakdown. Dashed lines mark 144 Hz (6.9 ms) and 60→120 FPS (16.7 ms) budgets.

Nano over RIFE (0.9525 vs. 0.9515) suggests that depthwise separable convolutions preferentially preserve structural edges, which dominate perceptual quality in gaming content.

B. Practical Deployment

Open-Frame-gen is a standalone Windows executable with a modular C++ design, comprising three zero-dependency components. The CaptureEngine uses the DirectX Desktop Duplication API to map the compositor back-buffer as an ID3D11Texture2D directly into CUDA memory via `cudaGraphicsD3D11RegisterResource`, eliminating $O(N)$ CPU copy overhead. The InferenceEngine wraps TensorRT with dual input buffers for asynchronous capture and inference, optimized for the `sm_86` (Ampere) architecture. The OverlayWindow combines GDI+ and D3D11, using `SetWindowDisplayAffinity` to block recursive capture and `DwmFlush()` for V-Sync alignment.

Unlike VFIMamba’s Linux-only `mamba-ssm` dependency, Open-Frame-gen uses only standard TensorRT and ONNX operations. Unlike NVIDIA DLSS Frame Generation or AMD Fluid Motion Frames, it requires no per-game integration, runs on any CUDA-capable GPU regardless of generation, and is fully open-source.

VIII. LIMITATIONS AND FUTURE WORK

Platform Dependency. The system is currently Windows-only due to its use of DirectX Desktop Duplication and Win32 compositor APIs. Linux support would require PipeWire/XSHM capture and X11/Wayland overlay integration; the neural model itself is platform-agnostic.

Known Failure Modes. Single-pass motion estimation produces ghosting during rapid camera movements, and occluded regions are poorly inpainted. Thin structures below the H/8 feature pyramid resolution may vanish, and stateless per-frame processing causes temporal jitter on static UI elements.

Future Directions. INT8 quantization could reduce inference latency to 3–5 ms. Knowledge distillation from RIFE as a teacher model may close the 0.87 dB PSNR gap to approximately 0.5 dB while preserving throughput.

IX. CONCLUSION

Open-Frame-gen demonstrates that temporal fluidity can be prioritized over spatial fidelity to achieve practical real-time VFI on constrained hardware. The Nano architecture reaches 101.1 FPS at 1080p with only $\sim 90\text{K}$ parameters, combining depthwise separable convolutions and channel attention to reduce complexity without sacrificing structural coherence. Compared to RIFE HDv3, Nano achieves a $12.1\times$ latency reduction and $10\times$ lower VRAM usage, enabling smooth frame generation for gaming, AR/VR, and video conferencing on entry-level GPUs. Released as an open-source framework, it provides a foundation for further research in efficient, deployment-ready video manipulation.

REFERENCES

- [1] Z. Huang, T. Zhang, W. Heng, B. Shi, and S. Zhou, “Real-Time Intermediate Flow Estimation for Video Frame Interpolation,” in *Proc. ECCV*, 2022.
- [2] G. Zhang, C. Liu, Y. Cui, X. Zhao, K. Ma, and L. Wang, “VFIMamba: Video Frame Interpolation with State Space Models,” in *Proc. NeurIPS*, 2024.
- [3] Z. Xiang, Z. Liu, M. Zhou, Y. Fan, and J. Chen, “LPVFI: Lightweight and Progressive Video Frame Interpolation with Dynamic Computation Allocation,” *arXiv:2502.03567*, 2025.
- [4] S. Niklaus, L. Mai, and F. Liu, “Video Frame Interpolation via Adaptive Separable Convolution,” in *Proc. ICCV*, 2017.
- [5] W. Bao *et al.*, “Depth-Aware Video Frame Interpolation,” in *Proc. CVPR*, 2019.
- [6] L. Kong *et al.*, “IFRNet: Intermediate Feature Refine Network for Efficient Frame Interpolation,” in *Proc. CVPR*, 2022.
- [7] A. G. Howard *et al.*, “MobileNets: Efficient CNNs for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [8] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *Proc. CVPR*, 2018.
- [9] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” *NeurIPS Deep Learning Workshop*, 2015.
- [10] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *Proc. CVPR*, 2018.
- [11] K. Soomro, A. R. Zamir, and M. Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in The Wild,” *arXiv:1212.0402*, 2012.
- [12] S. Baker *et al.*, “A Database and Evaluation Methodology for Optical Flow,” *IJCV*, vol. 92, no. 1, 2011.
- [13] T. Xue, B. Chen, J. Wu, D. Wei, and W. T. Freeman, “Video Enhancement with Task-Oriented Flow,” *IJCV*, vol. 127, no. 8, pp. 1106–1125, 2019.
- [14] A. B. Watson, A. J. Ahumada Jr., and J. E. Farrell, “The window of visibility: a psychophysical theory of fidelity in time-sampled visual motion displays,” *JOSA A*, vol. 3, no. 3, pp. 300–307, 1986.